



Introducción a la Visión Computacional (II)

Ing. Jaime Antonio Huaytalla Pariona
jhuaytalla@uncp.edu.pe

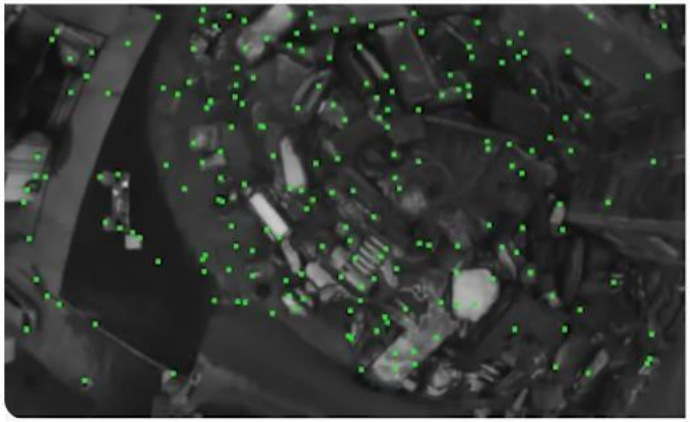
Temas

1. **Detectores y Descriptores**
2. Correspondencia de Imágenes
3. Detección de Objetos

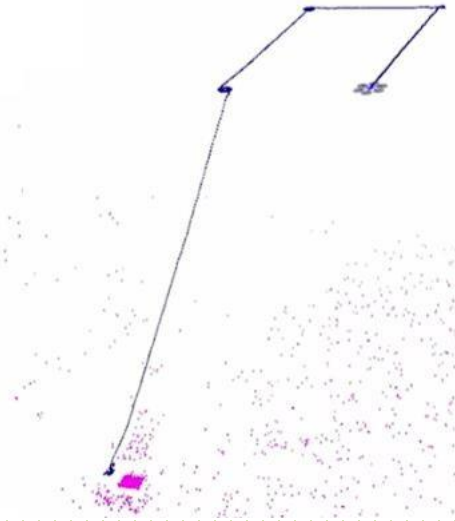
¿Para qué detectores y descriptores?

- Odometría visual: Estimación de la posición/orientación analizando los cambios que el movimiento genera en las imágenes de la cámara
  - Se encuentra “keypoints” (puntos de interés) en cada imagen
  - Se hace seguimiento de los “keypoints” en las imágenes

Imagen de la cámara



https://youtu.be/2YnIMfw6bJY



Posición del robot

- Detectores: permiten encontrar “keypoints” (puntos característicos)
- Descriptores: caracterizan cada “keypoint”

Forster et al. (2017) SVO: Semi-Direct Visual Odometry for Monocular and Multi-Camera Systems

¿Para qué detectores y descriptores?

Odometría Visual

- Principales etapas

Secuencia de imágenes

↓

Detección de puntos de interés ("features")

↓

Correspondencia entre puntos ("feature matching")

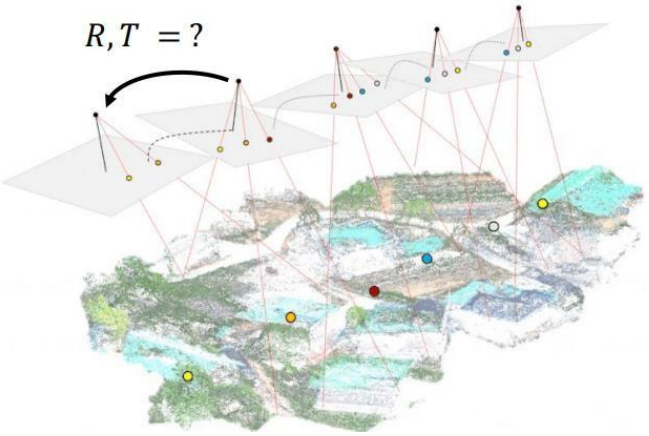
↓

Estimación del movimiento

↓

Optimización local

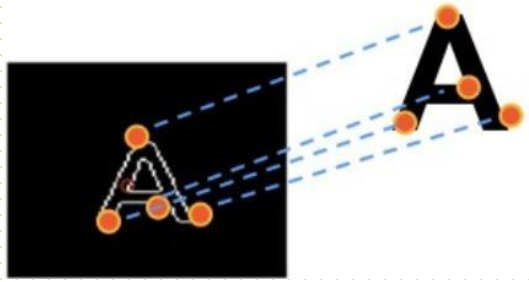


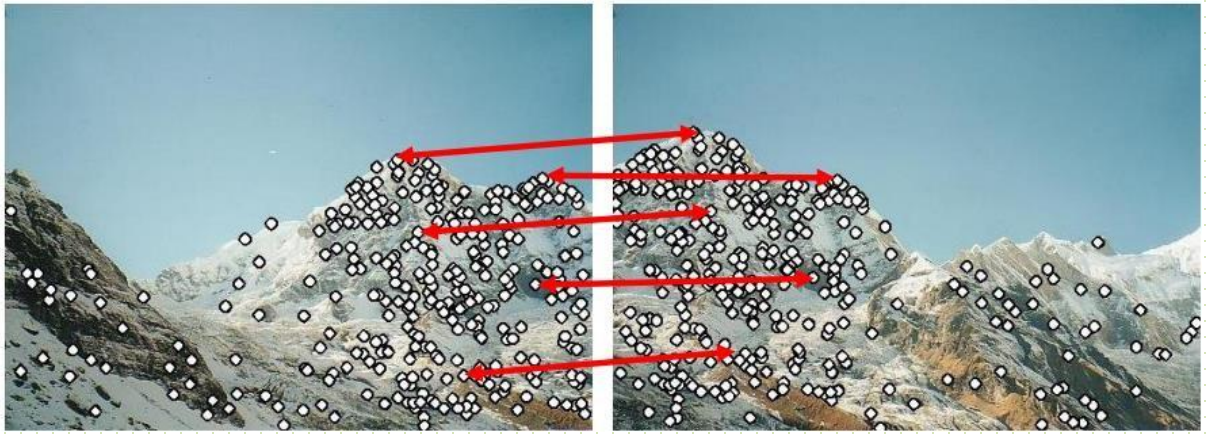


[Imágenes de Davide Scaramuzza, UZH]

¿Para qué detectores y descriptores?

- Feature Matching:
  - Encontrar puntos de interés (“keypoints”) correspondientes en dos imágenes
  - Aplicaciones:
    - Creación de panoramas
    - Reconocimiento de objetos
    - Reconstrucción 3D
    - Reconocimiento de lugares
    - Indexamiento en bases de datos (http://tineye.com o Google Images)



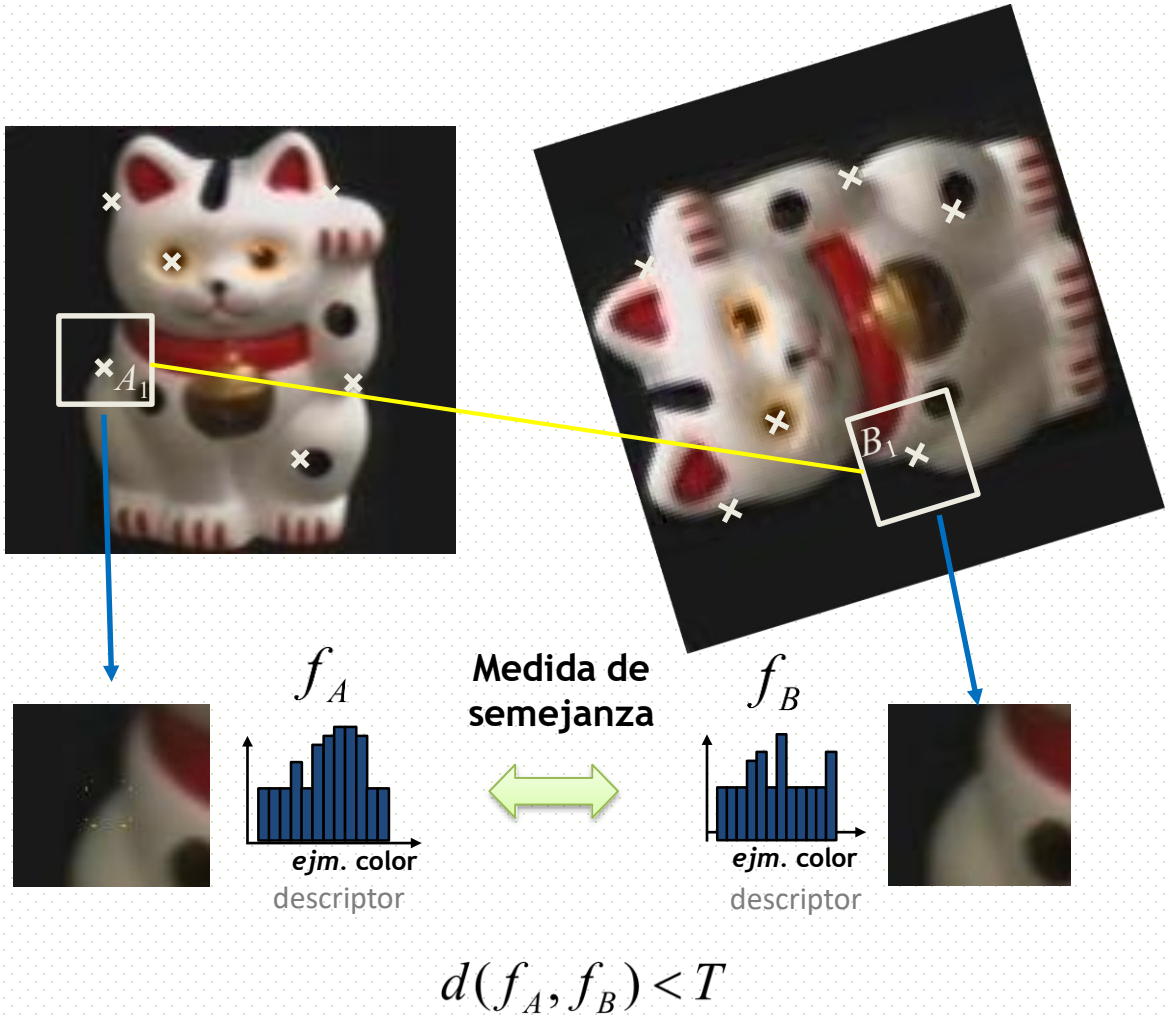


Feature Matching: Enfoque General

1. Encontrar puntos de interés (*keypoints, "detectores"*)

2. Obtener un *descriptor* local (vector) para la región alrededor de cada punto de interés

3. *"Matching"*: encontrar correspondencias entre descriptores locales basados en semejanza



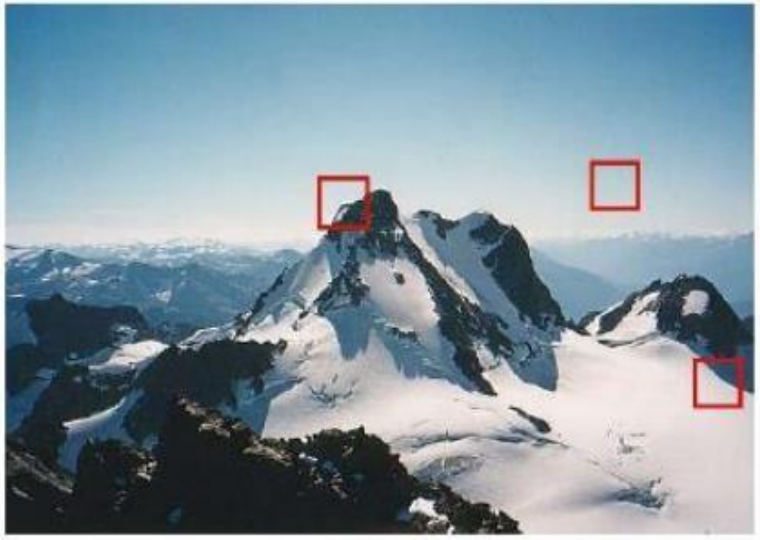
$$d(f_A, f_B) < T$$

Detectores

¿Qué es un buen detector?

- Un detector debe ser fácilmente reconocible (“invariante”)
  - ¿Qué regiones son más fácilmente identificables?

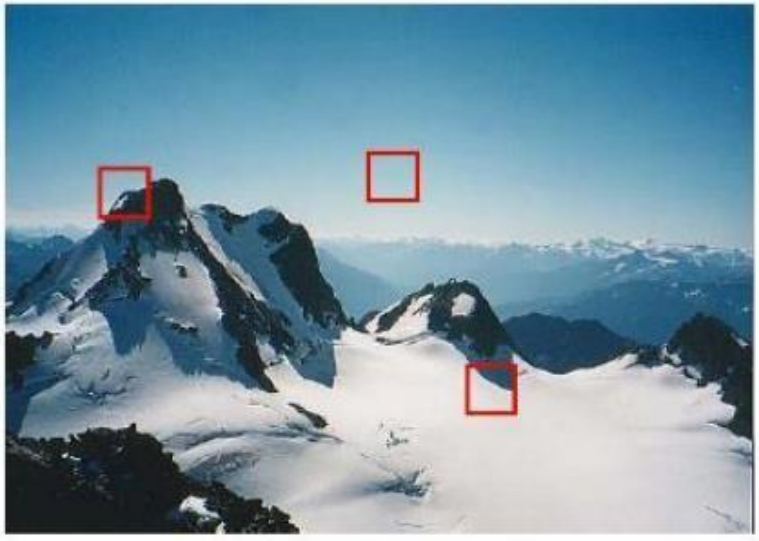
Imagen 1





A B C

Imagen 2





A B C

Detectores

Tipos de Puntos de Interés

- Esquina ("***corner***")
  - Es la intersección de dos o más bordes
  - Tienen alta exactitud de localización (adecuados para odometría visual)
  - Son menos distinguibles que los "blobs"



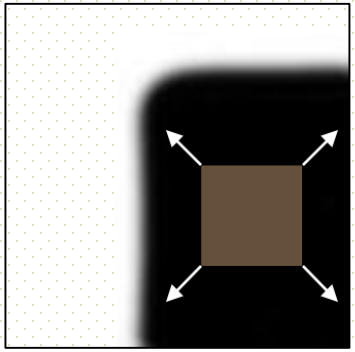
  - Detectores de esquinas: Moravec, Harris, Shi-Tomasi, SUSAN, FAST, etc.

Detectores

Tipos de Puntos de Interés

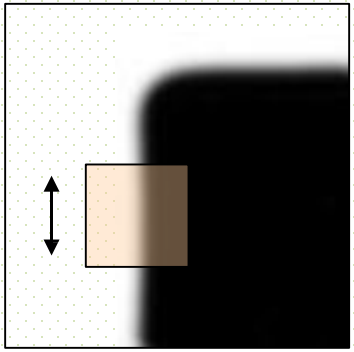
- Esquina (“***corner***”)
  - Debe ser reconocible localmente (en una pequeña ventana)
  - Un desplazamiento de la ventana en cualquier dirección debe dar una gran variación de intensidad

Región “plana”



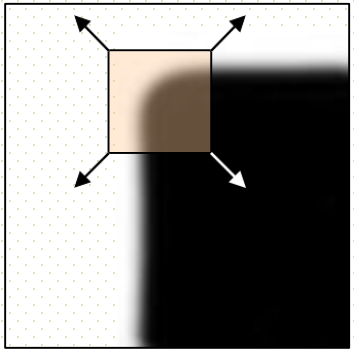
Igual en todas direcciones

“Borde”



No cambia en la dirección del borde

“Esquina”



Cambia en todas las direcciones

Detectores

Tipos de Puntos de Interés

- Blob
  - Es cualquier otro patrón que no sea una esquina y que difiera significativamente de los vecinos
  - Tienen menos exactitud de localización (que las esquinas)
  - Son más distinguibles que las esquinas (adecuados para reconocimiento de lugares)



  - Detectores de “blobs”: MSER, LOG, DoG (SIFT), SURF, CenSurE

Detectores

Detector de Esquinas de Harris/Shi-Tomasi

- Algoritmo

1. Calcular las derivadas I_x, I_y (por ejemplo usando Sobel)
2. Calcular I_x^2, I_y^2, $I_x I_y$
3. Realizar la convolución de I_x^2, I_y^2, $I_x I_y$ con un filtro (Gausiano o de media) para obtener $\sum I_x^2$, $\sum I_y^2$, $\sum I_x I_y$ que son las entradas a la matriz M

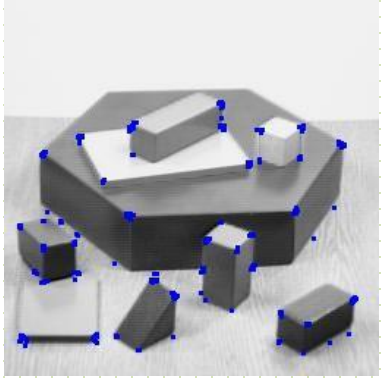
$$M = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

4. Calcular la métrica de esquina R según Harris o Shi-Tomasi

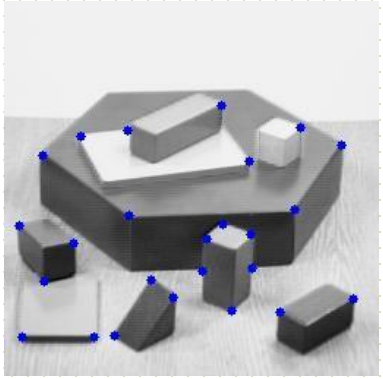
$$R = \min(\lambda_1, \lambda_2)$$ Shi-Tomasi

$$R = \det(M) - k\ \mathrm{tr}(M)$$ Harris

5. Encontrar puntos con respuesta fuerte (R > umbral)
6. Tomar los máximos locales



Esquinas con Harris

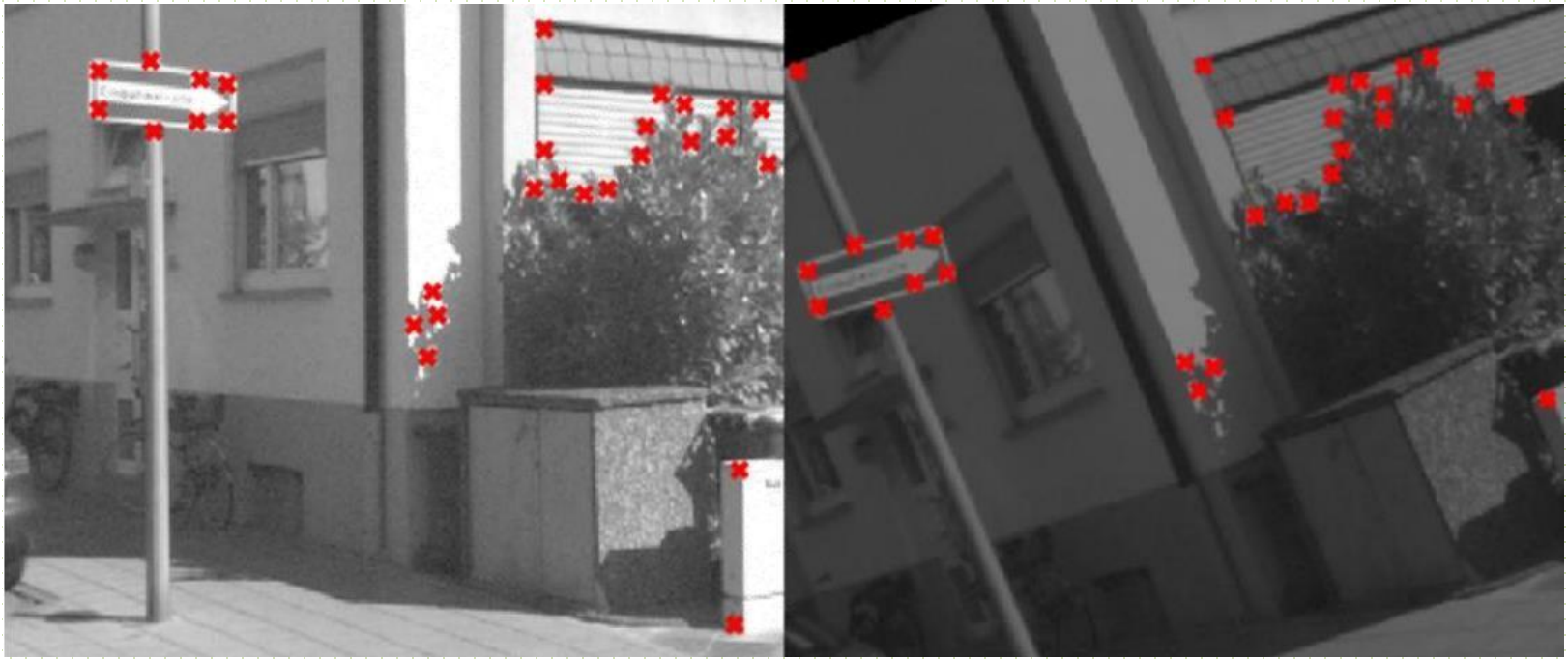


Shi-Tomasi

Detectores

Detector de Esquinas de Harris/Shi-Tomasi

- Ejemplo
  - Esquinas (en rojo) en las imágenes

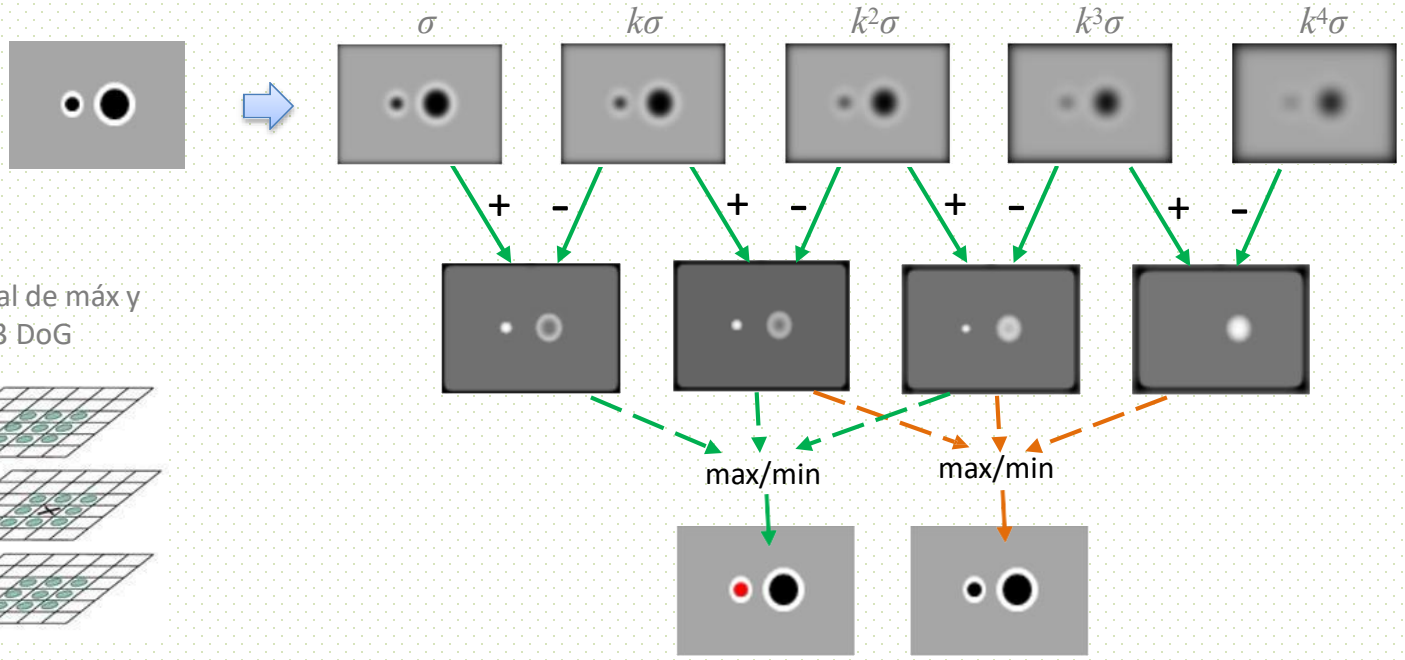


  - Notar la invarianza a cambios de color y transformaciones geométricas

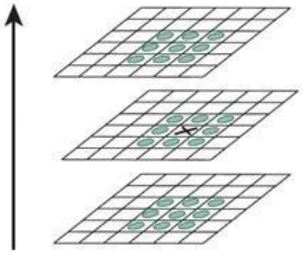
Detectores

Detector de Blobs usando DoG

- DoG (Diferencia de Gaussianos)
  - Con bajos σ: detecta características (cambios de intensidad) en partes pequeñas
  - Con altos σ: detecta características (cambios de intensidad) en partes grandes
- Puntos de interés: máximos y mínimos locales de 3 DoG consecutivos (por "robustez")



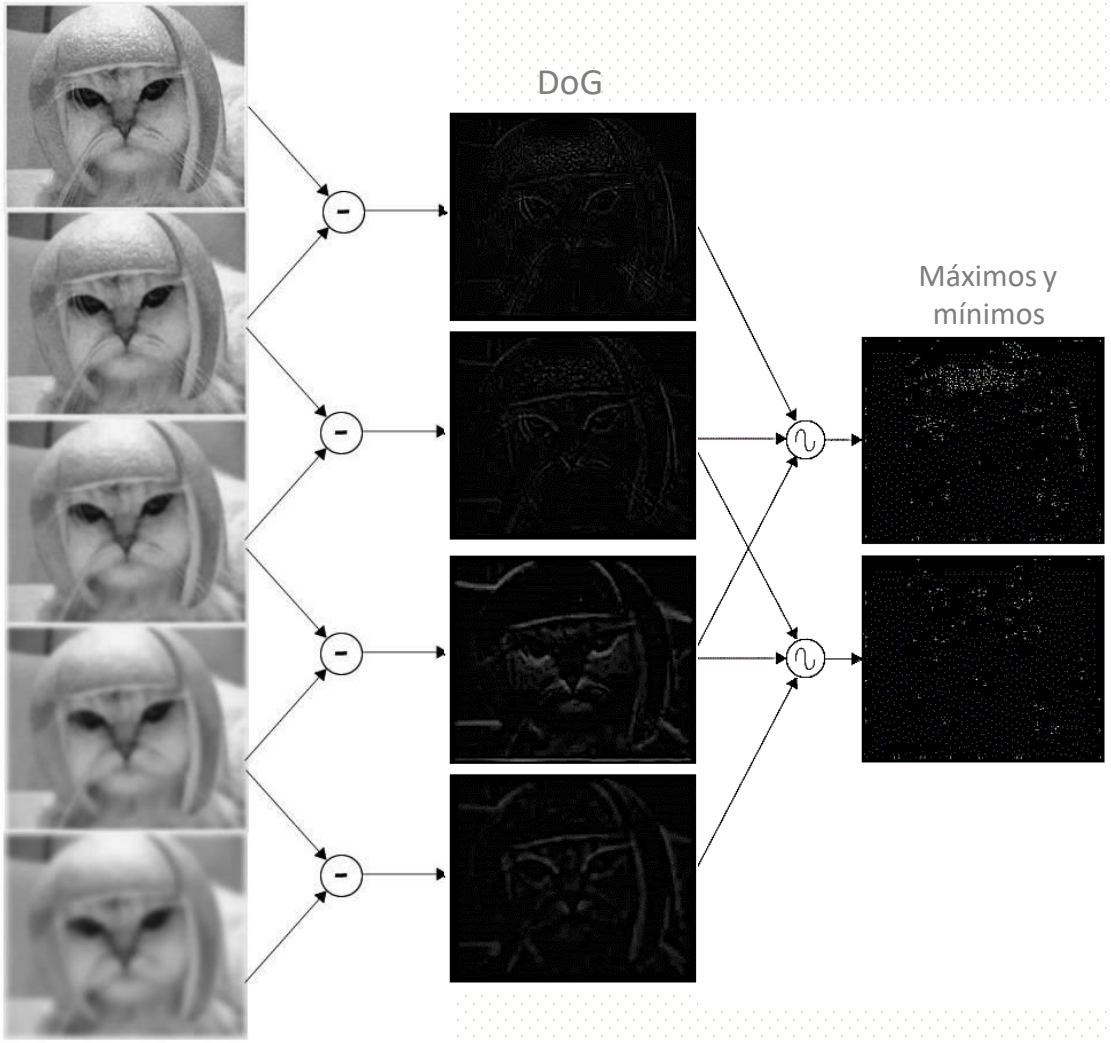
Búsqueda local de máx y min en 3 DoG



Detectores

Detector de Blobs usando DoG

- Ejemplo



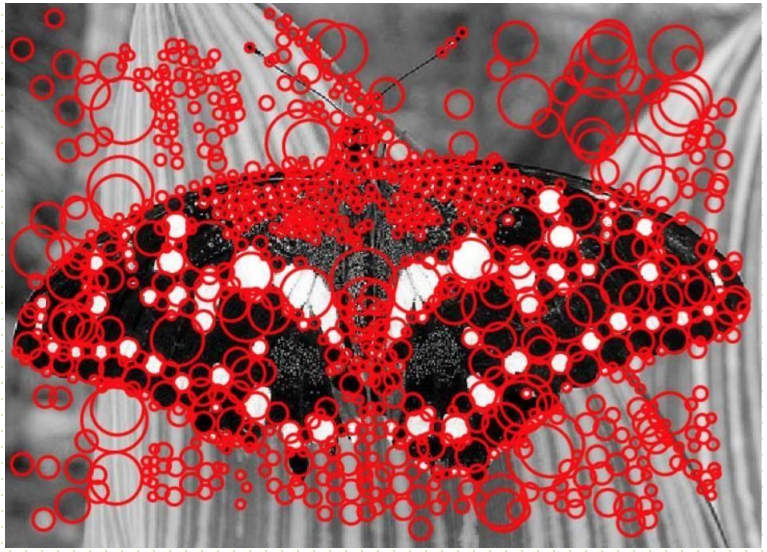
[aishack.in]

Detectores

Detector de Blobs usando DoG

- Ejemplo
  - Puntos detectados usando DoG



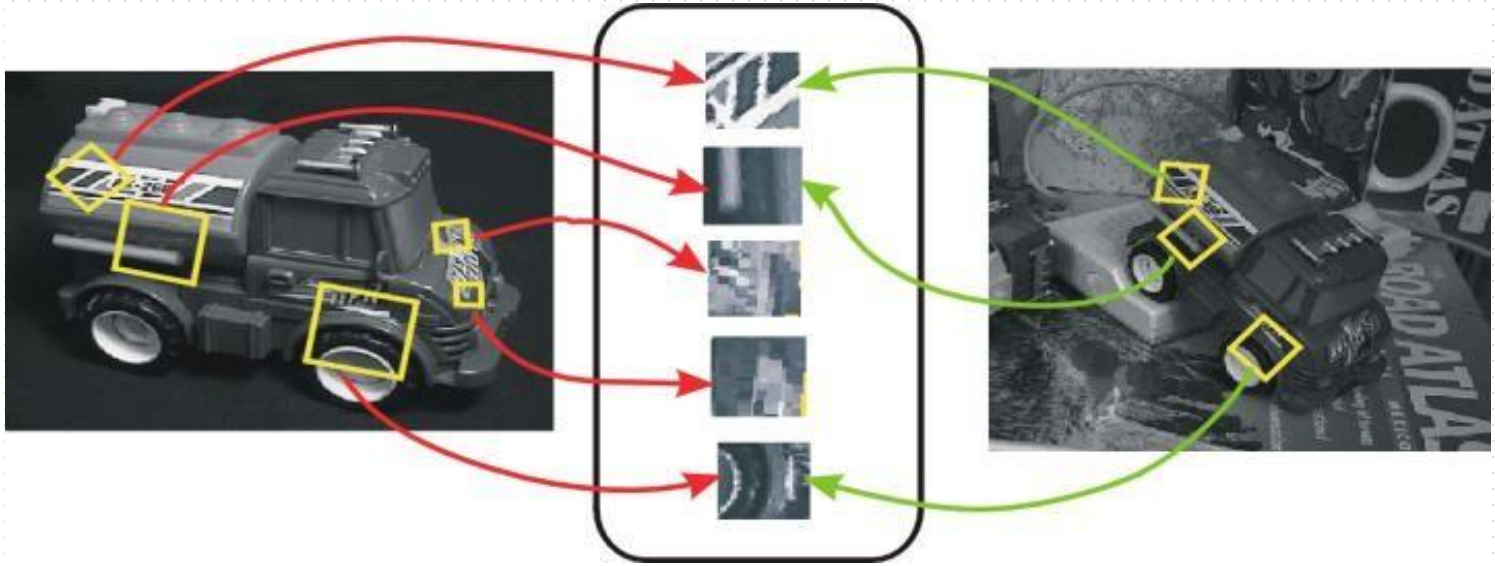


El tamaño de los círculos indica la escala en la que se detectó el punto de interés

Descriptores

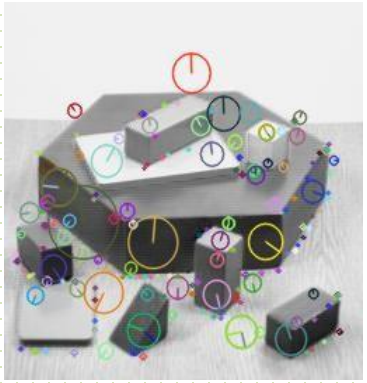
- ¿Qué es?
  - Es un vector que contiene las características de una región alrededor de un punto de interés (identificado con un detector)
  - "Describe" al punto identificado con el detector (su vecindad)

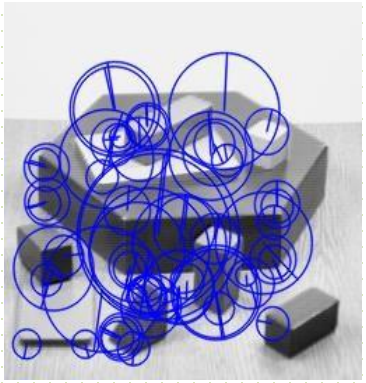
- Uso
  - Obtener correspondencias de puntos de interés ("keypoints") entre imágenes

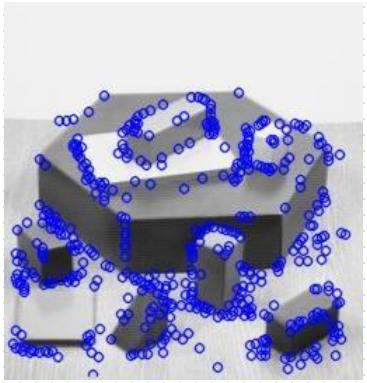


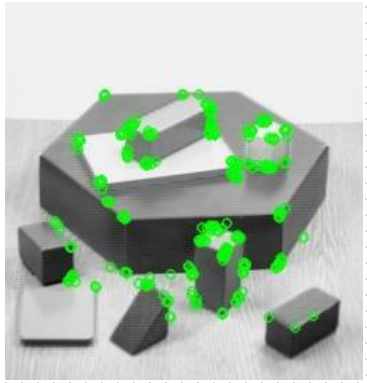
Descriptores

- Principales descriptores:
  - *SIFT*: **S**cale-**I**nvariant **F**eature **T**ransform
  - *HOG*: **H**istogram of **O**riented **G**radients
  - *SURF*: **S**peeded-**u**p **R**obust **F**eatures (detector y descriptor)
  - *BRIEF*: **B**inary **R**obust **I**ndependent **E**lementary **F**eatures (aplicable a esquinas o blobs)
  - *ORB*: **O**riented FAST and **R**otated **B**RIEF (aplicable a esquinas o blobs)
  - *BRISK*: **B**inary **R**obust **I**nvariant **S**calable **K**eypoints (aplicable a esquinas o blobs)
  - *FREAK*: **F**ast **RE**tin**A** **K**eypoint (aplicable a esquinas o blobs)
  - *MOPS:* **M**ulti-scale **O**riented **P**atche**S**
  - *MSER:* **M**aximally **S**table **E**xtremal **R**egions


SIFT


SURF

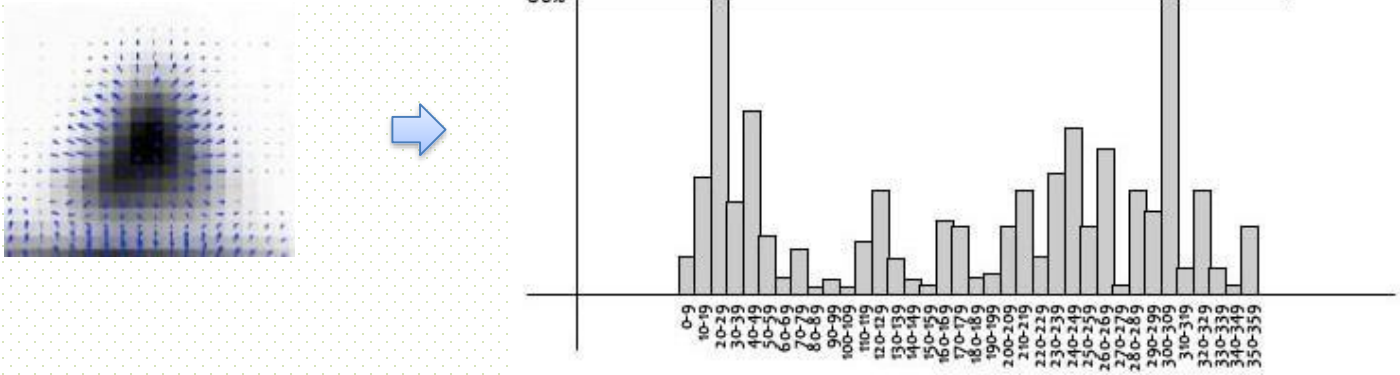

FAST


ORB

Descriptores

SIFT

- Utiliza los puntos de interés obtenidos con el detector DoG piramidal
- Obtiene una descripción de cada punto por separado
- Etapa 1: "Estandarizar" la orientación
  - Se realiza un histograma (de 36 barras) de la "orientación del gradiente" alrededor del punto de interés



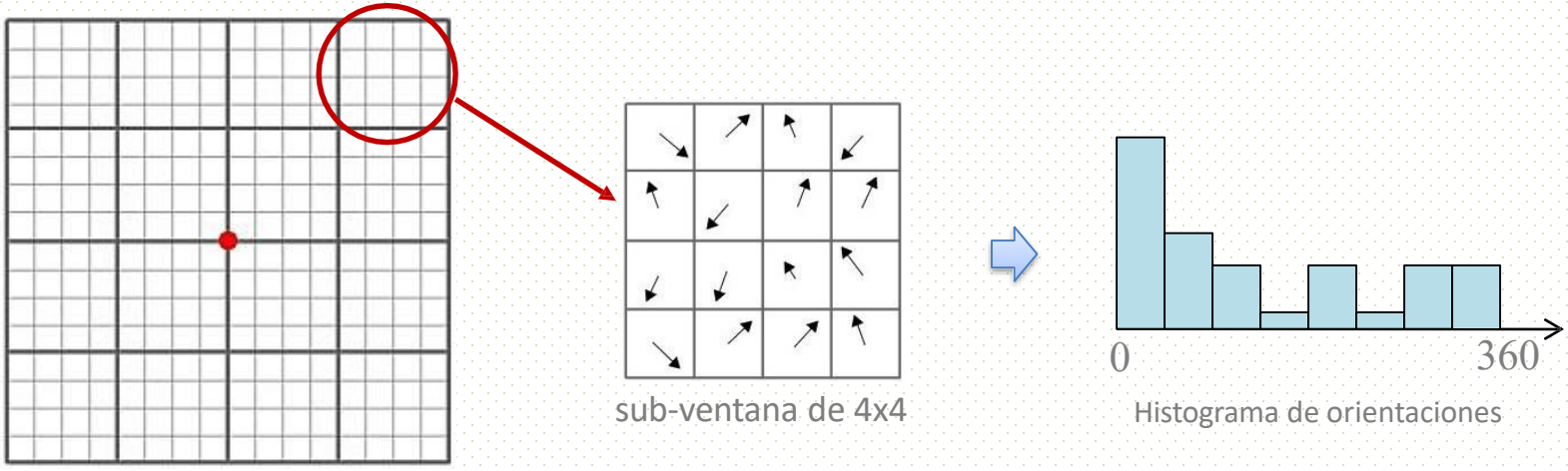
  - La orientación (θ) es el máximo del histograma: se "rota" según θ

[Imagen de: aishack.in/tutorials/sift-scale-invariant-feature-transform-introduction]

Descriptores

SIFT

- Etapa 2: Histograma de orientaciones
  - Se usa una ventana de 16x16 alrededor del punto de interés (“keypoint”)
  - La ventana se divide en sub-ventanas de 4x4 (16 sub-ventanas)



Ventana de 16x16 píxeles

sub-ventana de 4x4

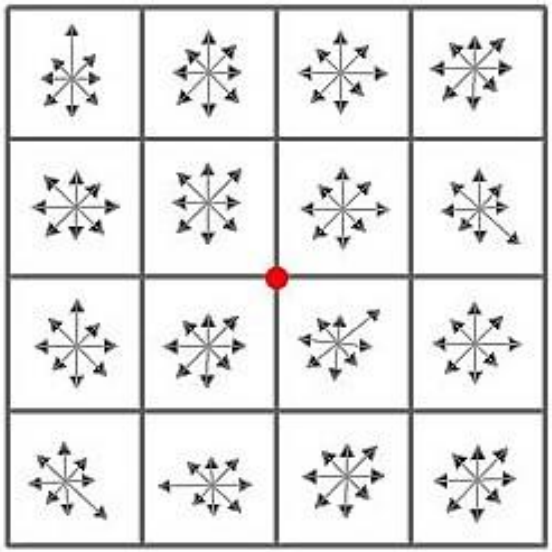
Histograma de orientaciones

  - En cada sub-ventana se calcula el histograma de orientaciones del gradiente
  - Se usa 8 orientaciones (8 “barras”)

Descriptores

SIFT

- Etapa 3: Construcción del descriptor
  - Cada punto de interés tiene 16 sub-ventanas y en cada una un histograma (de 8 orientaciones)



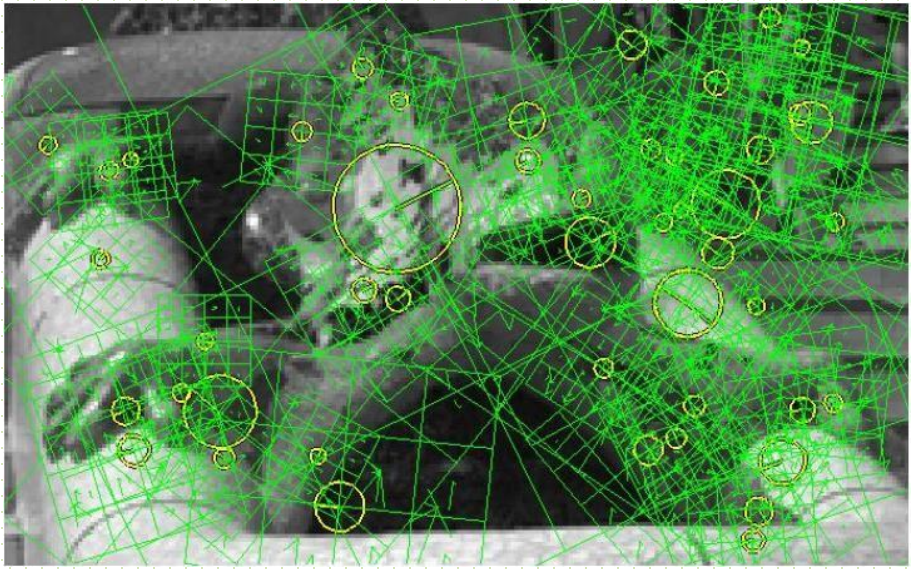
  - El descriptor contiene los valores de los histogramas: 16x8 = 128 valores
  - Descriptor: vector con 128 elementos (por cada punto de interés)

Descriptores

SIFT

- Detalles adicionales
  - Se pondera los píxeles de la ventana con una función Gaussiana
  - Se realiza interpolación “trilineal” (en x, y, θ) para la asignación del gradiente
  - Se normaliza el vector de 128 elementos

- Ejemplo



[Imagen de Ernest Valveny]

Descriptores

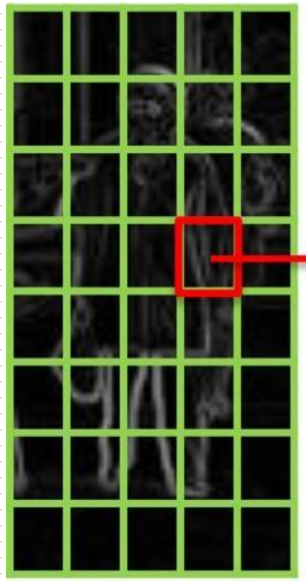
HOG

- Es un descriptor global de toda la imagen

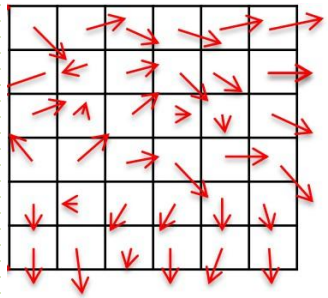
- Pasos
  - Se divide la imagen en celdas de tamaño fijo (usualmente de 6x6 u 8x8)
  - En cada celda se calcula el histograma de la orientación del gradiente (ponderado según su magnitud)



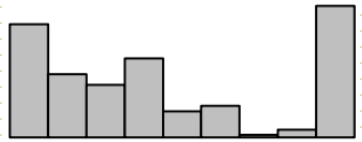
Imagen



Gradiente



Orientación del gradiente en 1 celda

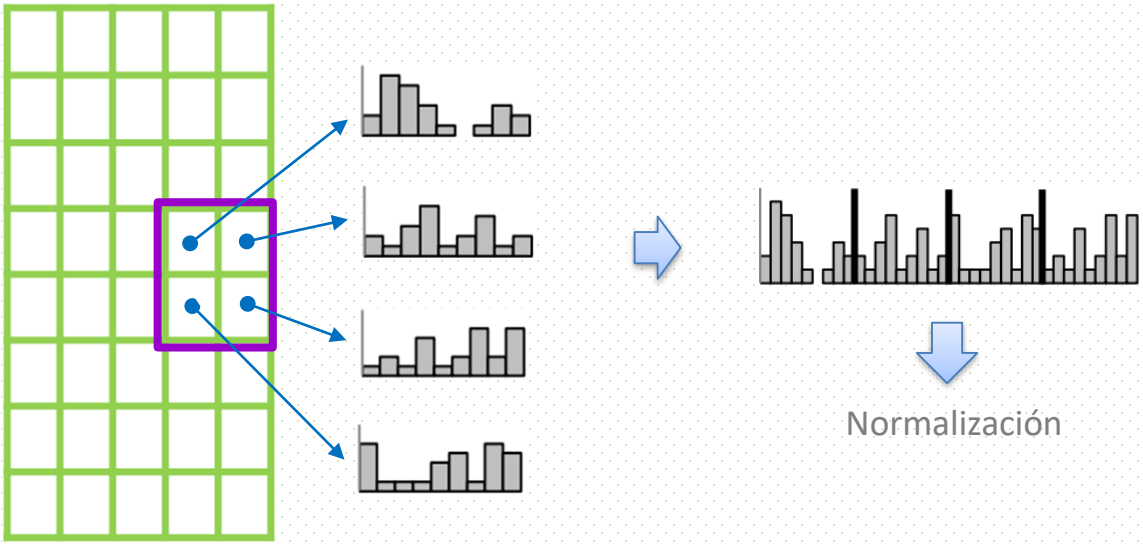


Histograma
(ejm. 9 intervalos)

Descriptores

HOG

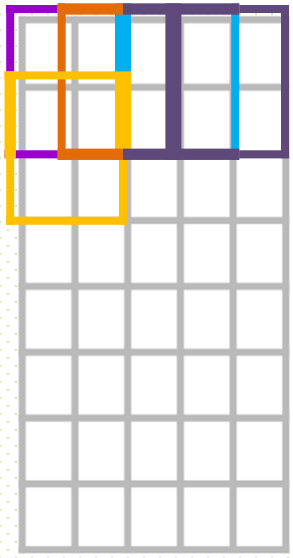
- Pasos
  - Normalización
    - Se agrupa celdas en “bloques” (ej. 2x2 celdas)
    - Se concatena los histogramas de cada bloque
    - Se normaliza los histogramas concatenados (de cada bloque) usando la norma L-2 (objetivo: minimizar variaciones de intensidad)

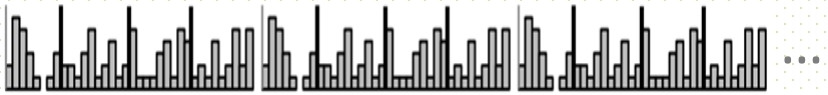


Descriptores

HOG

- Pasos
  - Concatenación de histogramas de boques (con solapamiento usualmente de 1 celda)





Descriptor final

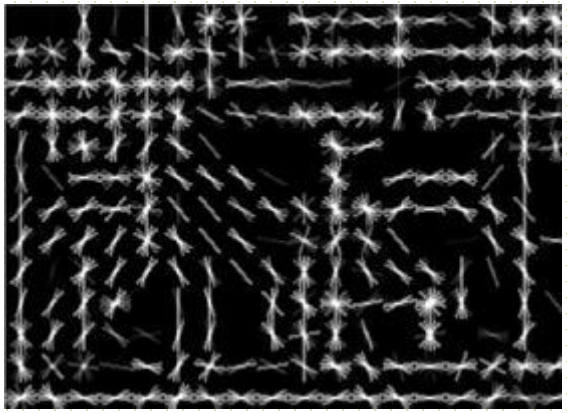
- Elementos en el descriptor = $n_b n_c n_i$ (n_b: núm. bloques, n_c: núm. celdas por bloque, n_i: número de elementos en cada histograma)
  - En el ejemplo: (28)(4)(9) = 1008 elementos

Descriptores

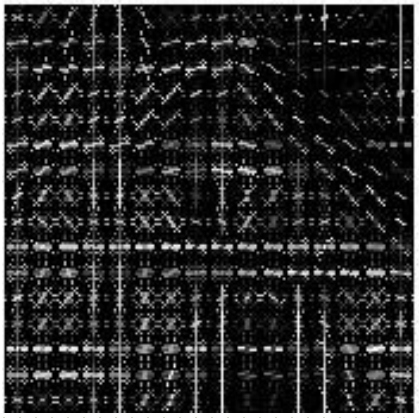
HOG

- Ejemplos





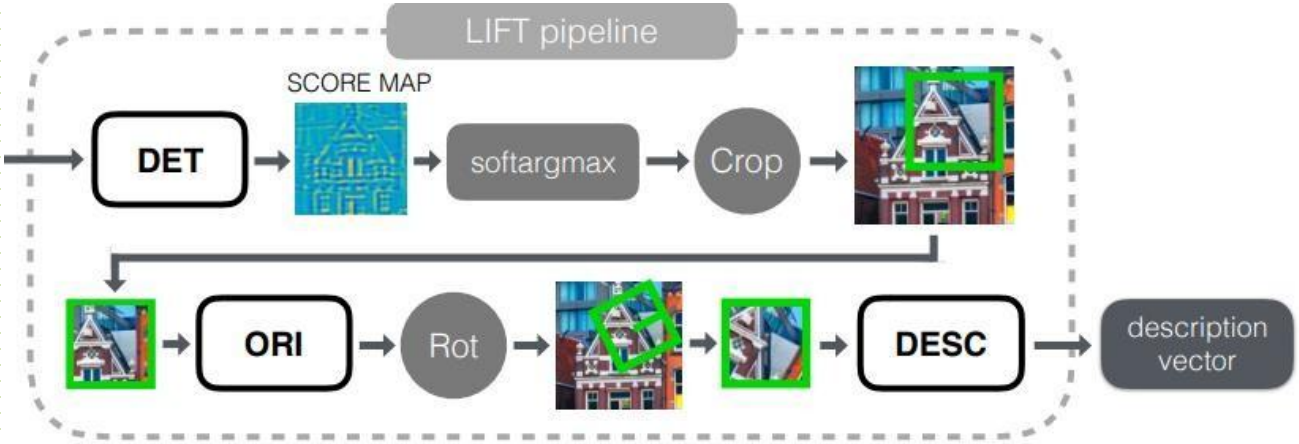




Descriptores

Basados en Deep Learning

- LIFT (Learned Invariant Feature Transform)
  - Es un detector y descriptor (entrenado por separado)
    - Una red neuronal predice la localización de un punto de interés
    - Otra red predice la orientación de la región alrededor del punto de interés (para "quitar" la rotación)
    - Otra red neuronal genera el descriptor de la región (128 elementos)
  - Es más repetible que SIFT

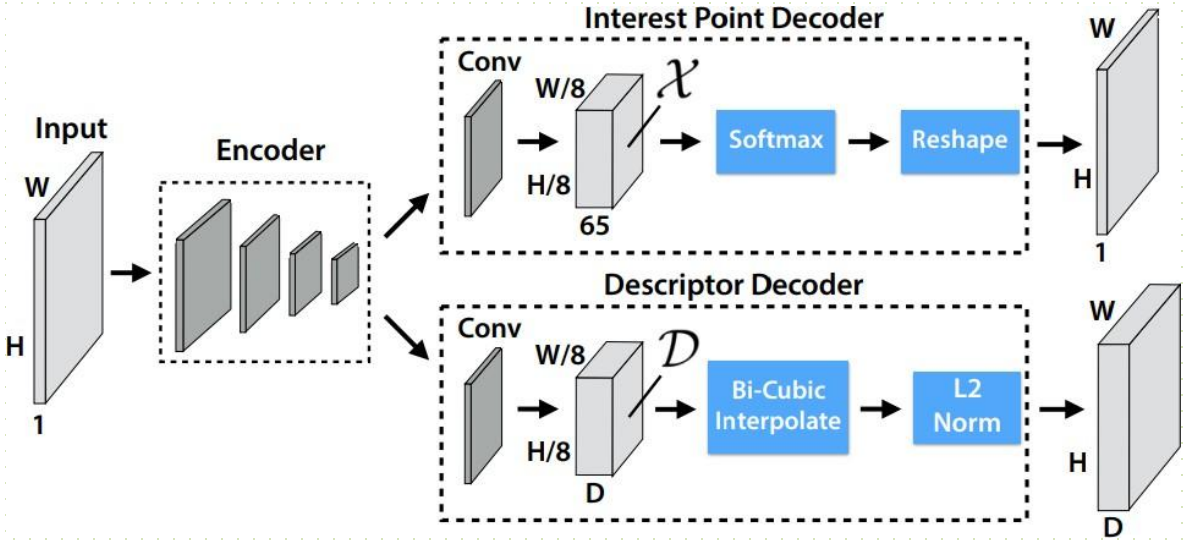


Yi et al. (2016), LIFT: Learned Invariant Feature Transform

Descriptores

Basados en Deep Learning

- SuperPoint
  - Es un detector y descriptor: aprende ambos en simultáneo
  - Es entrenado en imágenes sintéticas y se hace “fine tuning” en imágenes reales
  - Detector menos exacto que SIFT o LIFT
  - Descriptor funciona mejor que SIFT o LIFT
  - Más lento que SIFT o LIFT



DeTone et al. (2018). SuperPoint: Self-Supervised Interest Point Detection and Description.

Comparación

Detector	Localization Accuracy of the detector	Descriptor that can be used	Efficiency	Relocalization & Loop closing
Harris	++++	Patch SIFT/LIFT BRIEF ORB BRISK FREAK	+++ + ++++ ++++ +++ ++++	+ +++++ +++ ++++ +++ ++++
Shi-Tomasi	++++	Patch SIFT BRIEF ORB BRISK FREAK	++ + ++++ ++++ +++ ++++	+ +++++ +++ ++++ +++ ++++
FAST	++++	Patch SIFT/LIFT BRIEF ORB BRISK FREAK	++++ + ++++ ++++ +++ ++++	+ +++++ +++ ++++ +++ ++++
SIFT	+++	SIFT	+	++++
SURF	+++	SURF	++	++++
SuperPoint	++	SuperPoint	+	+++++

[Davide Scaramuzza, University of Zurich]

Temas

1. Detectores y Descriptores
2. **Correspondencia de Imágenes**
3. Detección de Objetos

Template Matching

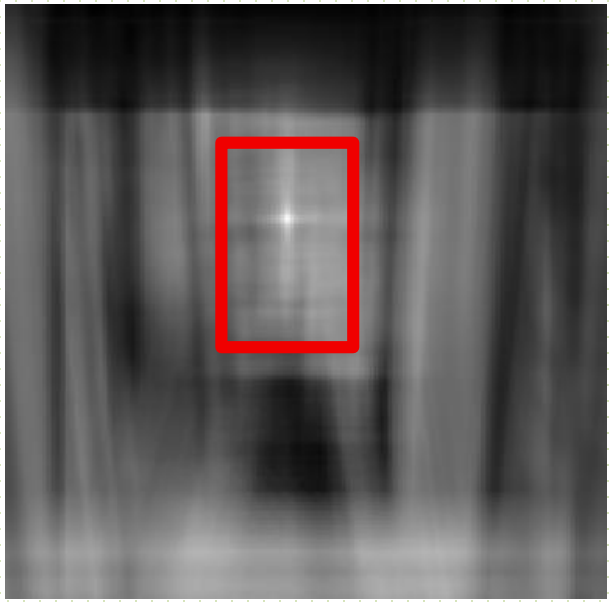
- Se le conoce como “correspondencia de patrones”
- Encuentra áreas similares a un “patrón” determinado
- Ejemplo: encontrar la silla



Silla



Imagen de prueba



Correlación cruzada

- Es la forma más simple (y menos precisa) de detectar un objeto en una imagen

Template Matching

- Se realiza la correlación del patrón con la imagen.
  - Métrica: correlación cruzada normalizada

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') \cdot I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

T: Template
I: Imagen
R: puntaje

  - Métrica: coeficiente normalizado

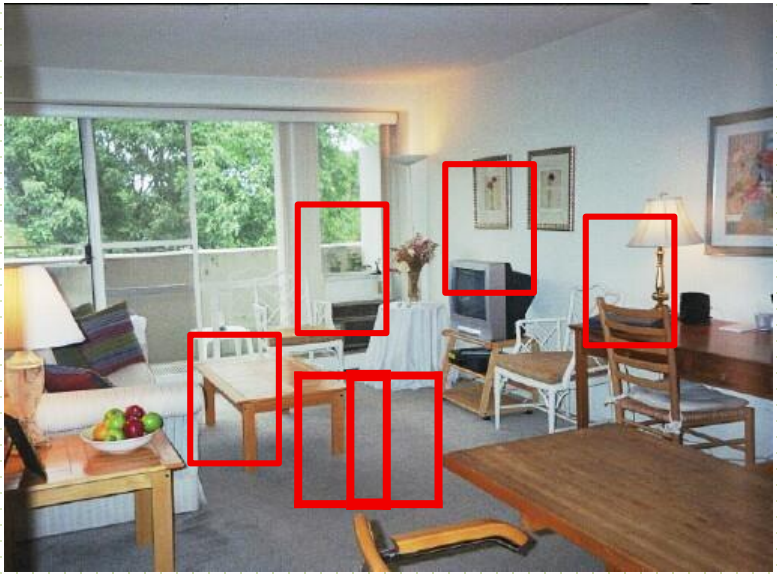
$$R(x,y) = \frac{\sum_{x',y'}(T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$$

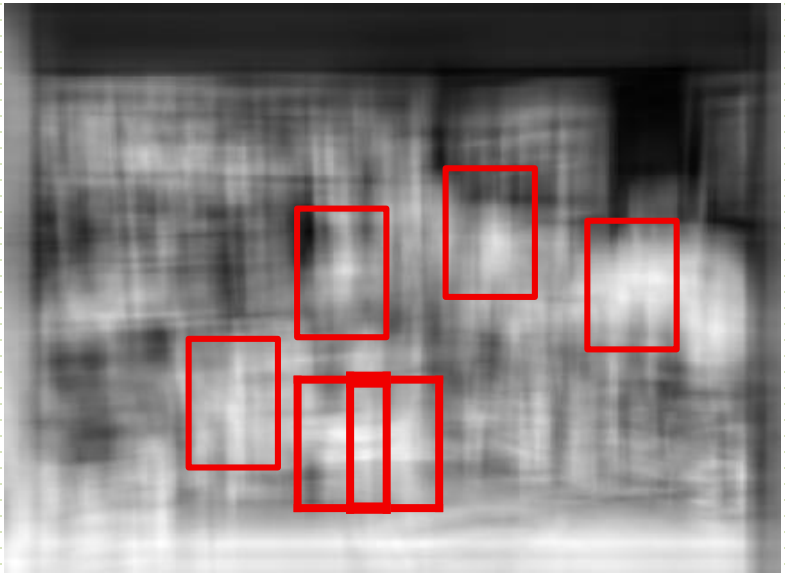
$$T'(x',y') = T(x',y') - 1/(w \cdot h) \cdot \sum_{x'',y''} T(x'',y'')$$
$$I'(x+x',y+y') = I(x+x',y+y') - 1/(w \cdot h) \cdot \sum_{x'',y''} I(x+x'',y+y'')$$

- Se reconoce el objeto cuando el valor de R es máximo
- También se puede usar un criterio para encontrar la “diferencia”

Template Matching

- Problemas:
  - Ejemplo: encontrar la silla  en la imagen

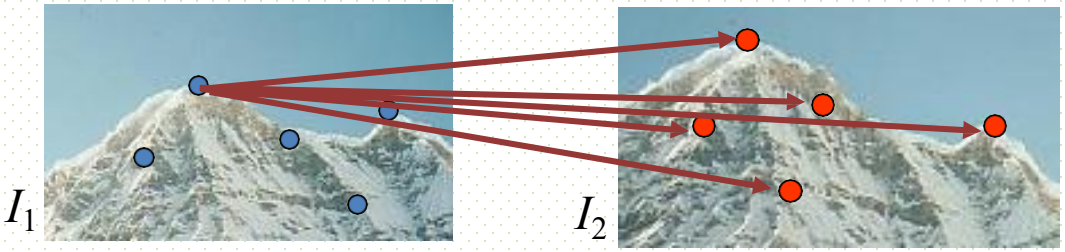




  - No funciona cuando hay cambios en ángulo desde donde se ve, cambios de iluminación, cambios en la articulación de las partes, oclusiones, etc.
  - No es un método robusto

Correspondencia de Imágenes

- Consiste en encontrar correspondencia entre descriptores
  - Dado un descriptor en I_1 … ¿a qué descriptor de I_2 corresponde?



- Requiere medir la distancia (d)
  - Distancia L_1 (SAD: Sum of Absolute Difference)
  - Distancia L_2 (SSD: Sum of Squared Difference)
  - Distancia de Hamming (XOR): solo si el descriptor es binario

- Fuerza bruta ("brute force")
  - Calcular d de cada descriptor de I_1 a todos los descriptores de I_2
  - Correspondencia: descriptor de I_2 con menor distancia (d)
  - Descartar distancias mayores a cierto umbral (si $d > d_{th}$)

Correspondencia de Imágenes

- Usar la menor distancia no detecta “malas” correspondencias
- Alternativa
  - Usar NNDR (Nearest Neighbor Distance Ratio)

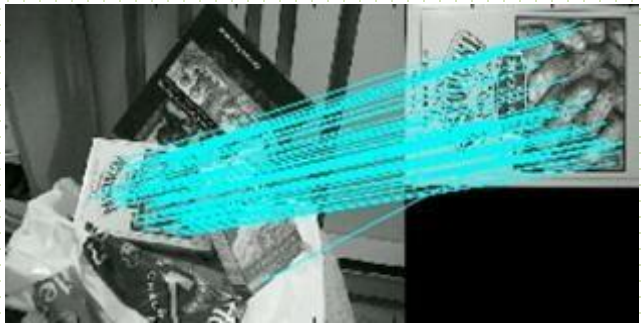
$$NNDR = \frac{d_1}{d_2}$$

d_1: distancia al punto más cercano de I_2

d_2: distancia al segundo punto más cercano de I_2

  - Si NNDR > 0.8: “mala” correspondencia
  - SI NNDR < 0.8: “buena” correspondencia

- Las correspondencias se pueden “refinar” (ejm. usando RANSAC para evitar “outliers”, o verificación cruzada), y hacer más rápidas (usando k-d tree)



Usando distancia Euclideana (L_2)

Correspondencia de Imágenes

Transformaciones

- Luego de detectar las correspondencias se puede "localizar" al objeto
  - Usando transformación afín
  - Usando transformación proyectiva (homografía)

- Transformación afín (T):

$$\underbrace{\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}}_{\text{Puntos en la imagen 1}} = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ 0 & 0 & 1 \end{bmatrix} \underbrace{\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}}_{\text{Puntos en la imagen 2}}$$

$$u = t_{11}x + t_{12}y + t_{13}$$
$$v = t_{21}x + t_{22}y + t_{23}$$

  - Términos:
    - t_{11}, t_{22}: escalado de x, y
    - t_{12}, t_{21}: "shearing"
    - t_{13}, t_{23}: traslación
  - Nota: t_{11}, t_{12}, t_{21}, t_{22} también pueden representar una rotación (con t_{13}=t_{23}=0)

Correspondencia de Imágenes

Transformaciones

- Para encontrar la transformación afin:

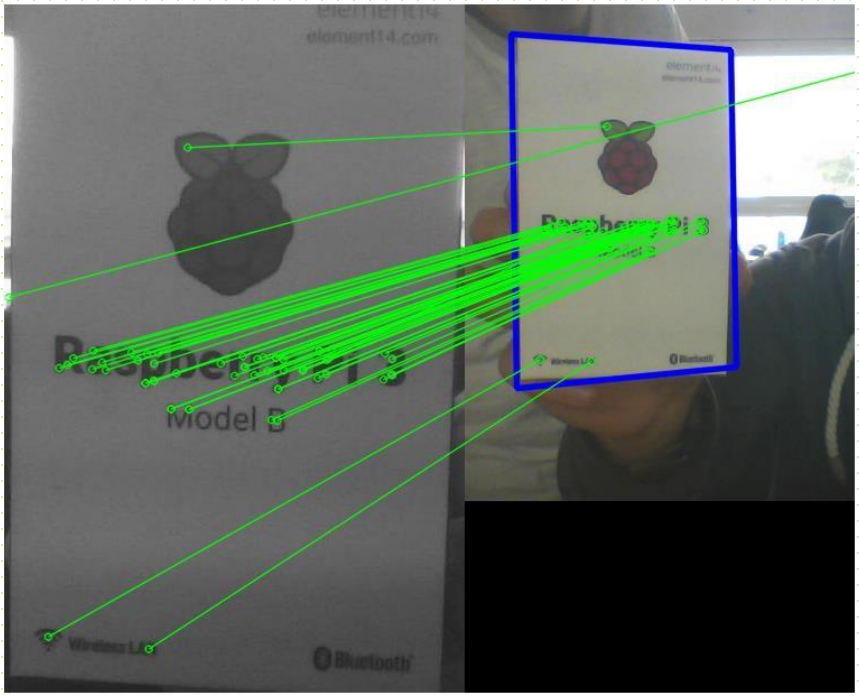
$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_1 & y_1 & 1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_2 & y_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} t_{11} \\ t_{12} \\ t_{13} \\ t_{21} \\ t_{22} \\ t_{23} \end{bmatrix} = \begin{bmatrix} u_1 \\ v_1 \\ u_2 \\ v_2 \\ \vdots \\ \vdots \end{bmatrix}$$

- Hay 6 variables (t_{11} … t_{23})
- Cada punto da 2 ecuaciones (para x, para y)
  - Se requiere al menos 3 puntos de correspondencia
- En la práctica se tiene más de 3 puntos:
  - Usar mínimos cuadrados (pseudo-inversa)

Correspondencia de Imágenes

Transformaciones

- Ejemplo:
  - Transformaciones entre objetos usando SIFT



Cuadro alrededor del objeto detectado

Correspondencia de Imágenes

Aplicación: Panorama

- Procedimiento
  - Encontrar puntos de interés y describirlos (por ejemplo usando SIFT)
  - Hacer una correspondencia entre descriptores (por ejemplo usando la distancia euclideana)
  - Encontrar la transformación (afín) entre imágenes: usar mínimos cuadrados (se requiere al menos 3 puntos)
  - RANSAC (descartar "outliers")



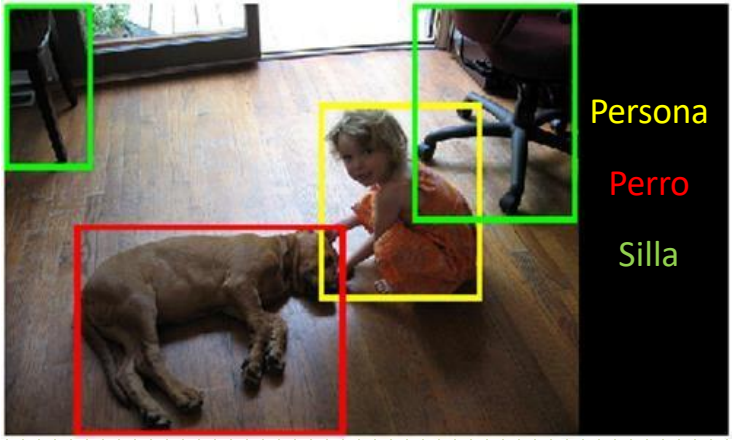
[Imagen de J.C. Niebles]

Temas

1. Detectores y Descriptores
2. Correspondencia de Imágenes
3. **Detección de Objetos**

Detección de Objetos

- Consiste en:
  - Detectar (localizar) dónde se encuentra algún objeto en una imagen
  - El objeto puede pertenecer a varias categorías (vehículos, personas, etc.)



- Dificultades:
  - Iluminación
  - Deformaciones
  - Ángulo de observación
  - Variaciones intra-clase

- Se puede usar la clasificación (saber si en una imagen hay un objeto) como una primera etapa

Detección de Objetos

Enfoques usuales

- Se suele utilizar:
  - Características locales
  - Ventanas deslizantes
  - Modelos de partes (deformables)
  - Bag of Features
  - Deep Learning

- Los enfoques tradicionales requieren:
  - Extracción de atributos (“features”)
  - Uso de Machine Learning para clasificación con base en los atributos







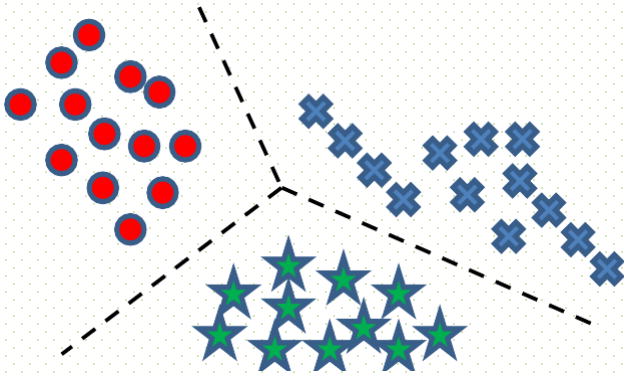


Entrenamiento basado en atributos
(SVM, redes neuronales, random forest, AdaBoost, etc.)

Detección de Objetos

Clasificación usando Machine Learning

- Métodos usuales
  - Support Vector Machines (SVM)
  - K-nearest neighbor
  - Naive Bayes
  - Árboles de decisión
  - Random Forest
  - Boosting (ejemplo: Ada Boost)
  - Redes neuronales (simples)



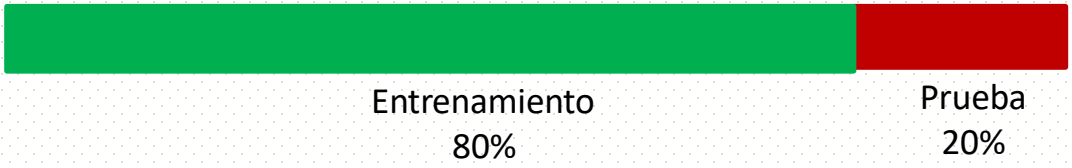
Clasificación de tres clases

- La evaluación se realiza con base en algunas métricas que se calculan a partir de una matriz de confusión
  - Métricas más usuales: exactitud, precisión, recall

Detección de Objetos

Uso de los Datos

- Los datos se suelen dividir en



- Conjunto de entrenamiento (training set)
  - Usado para aprender los parámetros del modelo (y determinar el mejor modelo)
  - Suele incluir un subconjunto de validación o uso de validación cruzada (con el que se “afina” el modelo para saber qué funciona mejor)
- Conjunto de prueba (test set)
  - Usado para realizar la prueba de desempeño final
  - NO se debe usar este conjunto para “afinar” el modelo entrenado
  - Se usa solo al final del proceso

Detección de Objetos

Esquema general de Clasificación

- Procedimiento
  - Se extrae características importantes ("features") usando descriptores (y sus respectivos detectores)
  - Usando estas características se entrena un clasificador
  - Usando el clasificador entrenado se predice la clase



Imágenes de entrenamiento

Detectores y descriptores

Entrenamiento

Clasificador entrenado



Imagen de prueba

Detectores y descriptores

Predicción

Detección de Objetos

Ventanas Deslizantes

- Procedimiento
  - Utilizar un descriptor y un clasificador (ejemplo [Dalal-Triggs]: HOG + SVM)
  - Deslizar una ventana por la imagen: en cada posición verificar si se encuentra el objeto deseado



Usando la misma ventana, ¿se podrá encontrar un ómnibus?

El tamaño de la ventana importa

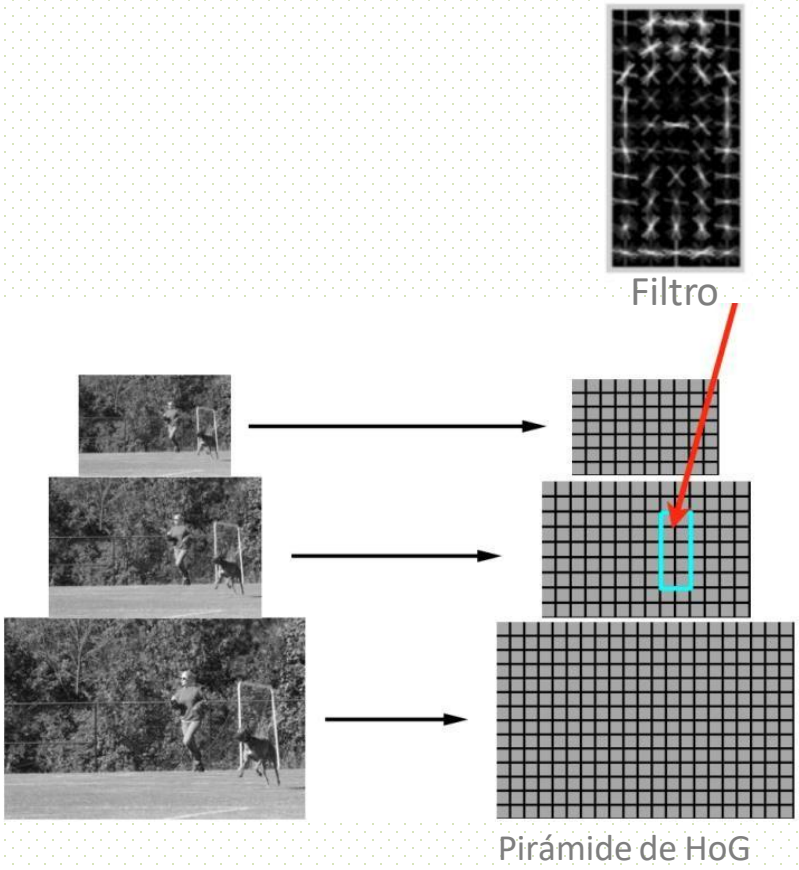
Se repite la búsqueda para ventanas de diferentes tamaños

Detección de Objetos

Ventanas Deslizantes

- Alternativa
  - No modificar el tamaño de la ventana
  - Modificar el tamaño de la imagen (multi-escala)

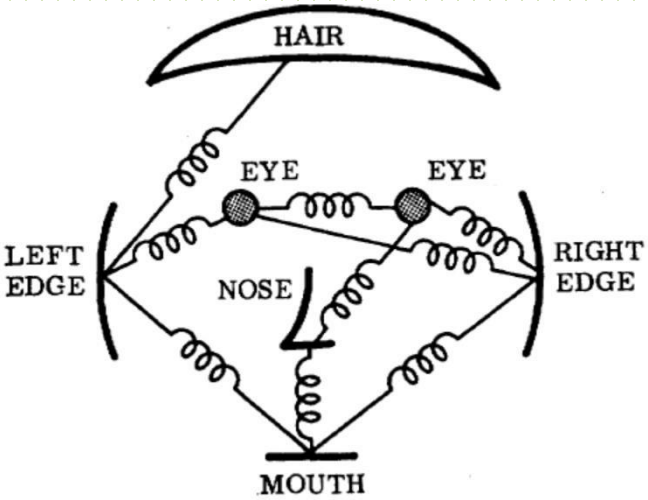


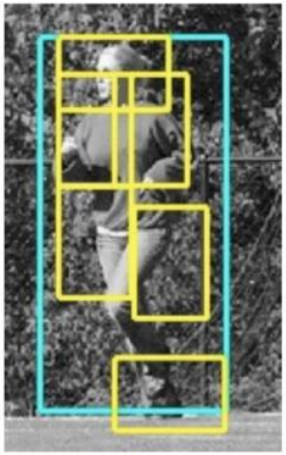


Detección de Objetos

Modelos de Partes Deformables

- Idea:
  - Representar un objeto como una colección de partes que tienen una configuración deformable
  - Hay conexiones tipo “resorte” (deformables) entre algunas de las partes





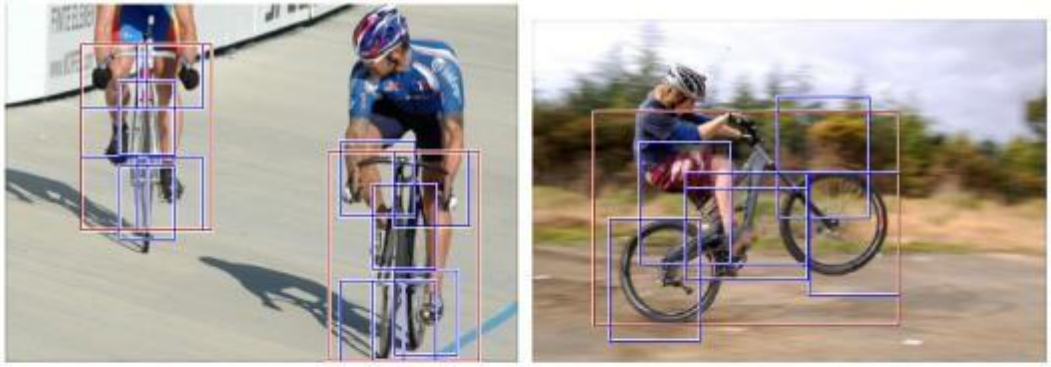
Ejemplo: cabeza, brazo derecho, brazo izquierdo, etc → persona

  - Los filtros de las partes tienen detalles más finos

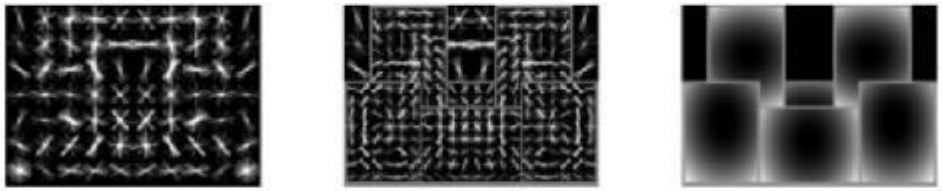
Detección de Objetos

Modelos de Partes Deformables

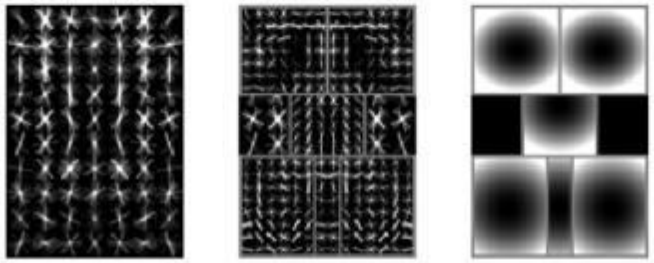
- Ejemplo:
  - Modelo de una bicicleta



Componente lateral



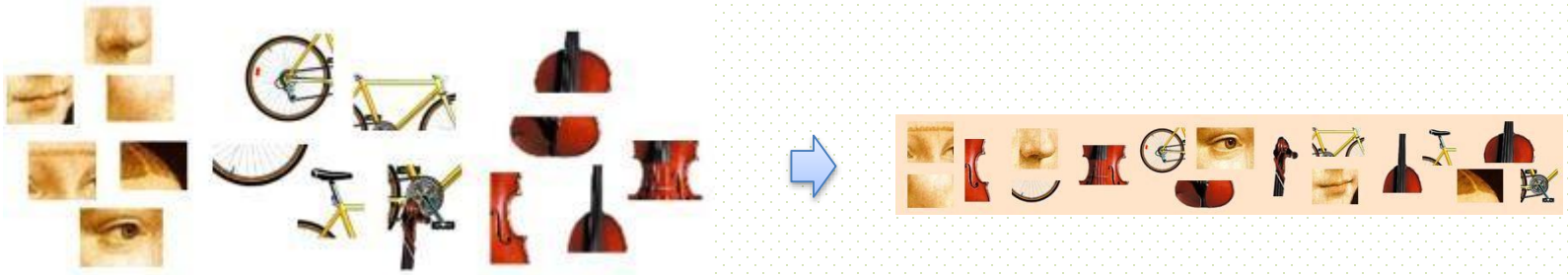
Componente frontal



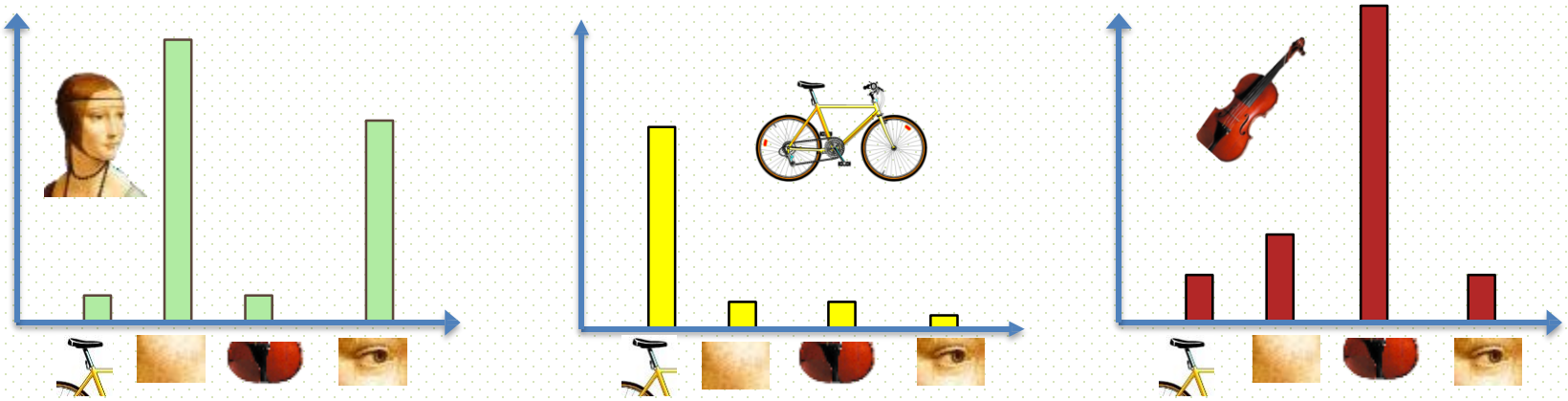
Detección de Objetos

Clasificación usando Bag of Features

- Consiste en:
  - Extraer características (“features”) de un conjunto de imágenes: “diccionario” o “vocabulario visual”



  - Representar los objetos por sus frecuencias de “palabras visuales”

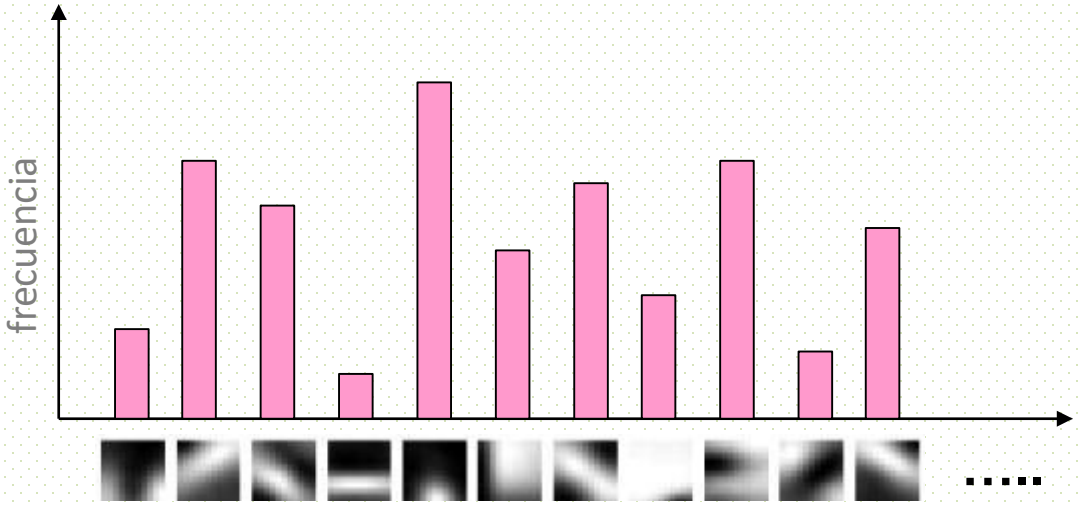


Detección de Objetos

Clasificación usando Bag of Features

- Ejemplo de representación de objeto

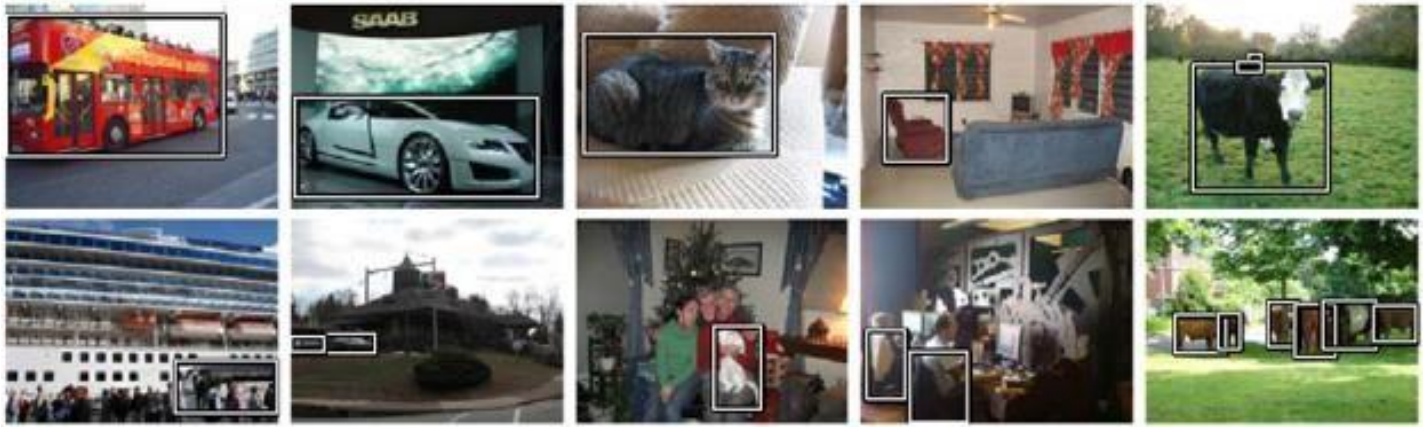




- Cada vector es una entrada a un clasificador
- Para reconocer un objeto se utiliza un clasificador entrenado

Detección de Objetos

- Bases de datos para entrenamiento
  - PASCAL VOC (Visual Object Classes) Challenge
    - http://host.robots.ox.ac.uk/pascal/VOC
    - 20 Categorías



  - ImageNet Large Scale Visual Recognition Challenge
    - 200 categorías
    - http://image-net.org/

Detección de Objetos

- Bases de datos para entrenamiento
  - CoCo (Common Objects in Context) challenge
    - https://cocodataset.org
    - 80 Categorías





Referencias

- R. Gonzales, R. Woods, *Digital Image Processing*. Pearson Edition, 2017
- E. Trucco, A. Verri, *Introductory Techniques for 3-D Computer Vision*. Prentice Hall, 1998. (*Capítulos 3-5*)
- R. Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2010 (*Capítulos 4, 14*): *http://szeliski.org/Book/*
- D. Forsyth, J. Ponce: *Computer Vision, A Modern Approach*, Pearson Education, NJ 2012 (*Capítulos* 5, 16, 17)
- R. Siegwart, R.I. Nourbakhsh, and D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT press, 2011. (*Capítulo 4*)